VALIDATION

Using Regular expression and Filterers

The PHP Filter Extension

- PHP filters are used to validate and sanitize external input.
- The PHP filter extension has many of the functions needed for checking user input, and is designed to make data validation easier and quicker.
- The filter_list() function can be used to list what the PHP filter extension offers

- **Example**
- ```
<table>
  <tr>
    <td>Filter Name</td>
    <td>Filter ID</td>
  </tr>
  <?php
  foreach (filter_list() as $id =>$filter)
{
      echo '<tr><td>' . $filter
. '</td><td>' . filter_id($filter)
.'</td></tr>';
  }
  ?>
</table>
```
```

Why Use Filters?

- Many web applications receive external input. External input/data can be:
  - User input from a form
  - Cookies
  - Web services data
  - Server variables
  - Database query results
- **You should always validate external data!** Invalid submitted data can lead to security problems and break your webpage!
  By using PHP filters you can be sure your application gets the correct input!

1. filter_var() Function

```
mixed filter_var( mixed $variable[,int $filter =FILTER_DEFAULT
[, mixed $options]])
```

The filter_var() function validate or sanitize data depending on the used filter.

- Validate (Determine if the data is in proper form) and
- sanitize data(Remove any illegal character from the data).

- The filter_var() function filters a single variable with a specified filter. It takes two pieces of data:
  - 1-The variable you want to check Note that scalar values are converted to string internally before they are filtered.
  - 2- The ID of the filter to apply. (check to use) If omitted, **FILTER_DEFAULT** will be used, which is equivalent to **FILTER_UNSAFE_RAW**. This will result in no filtering taking place by default.

- Return Values : Returns the filtered data, or **FALSE** if the filter fails.
- Filters and flags reference

Example (**filter_var**)

- EX : Validate an Integer

```
<?php    $int = 100;
if (!filter_var($int, FILTER_VALIDATE_INT) === false) {
    echo("Integer is valid");
} else {
    echo("Integer is not valid");
}?>
```

- EX : Validate an Email

```
<?php    email_a = 'joe@example.com'; $email_b = 'bogus';
if (filter_var($email_a, FILTER_VALIDATE_EMAIL)) {
  echo "This ($email_a) email address is considered valid.";
}
if (filter_var($email_b, FILTER_VALIDATE_EMAIL)) {
  echo "This ($email_b) email address is considered valid.";
} ?>
```

Example (**filter_var**)

- EX : Validate url

```php
<?php
if (!filter_var($url, FILTER_VALIDATE_URL) === false) {
    echo("$url is a valid URL");
} else {
    echo("$url is not a valid URL");
} ?>
```

- EX : Validate an IP

```php
<?php  $ip_a = '127.0.0.1'; $ip_b = '42.42';

if (filter_var($ip_a, FILTER_VALIDATE_IP)) {
   echo "This (ip_a) IP address is considered valid.";
}
if (filter_var($ip_b, FILTER_VALIDATE_IP)) {
    echo "This (ip_b) IP address is considered valid.";
}?>
```

Example (**filter_var**)

EX : Validate an username using REGEXP

```
<?php
if(filter_var($str, FILTER_VALIDATE_REGEXP,
array("options"=>array("regexp"=>"/^((\w){6,13})$/"))))
{
   echo("<br/>Variable value is  username");
} else {
   echo("<br/>Variable value is not a username");
} ?>
```

- EX : Validate an boalean var using FILTER_VALIDATE_BOOLEAN
- Returns TRUE for "1", "true", "on" and "yes"
- Returns FALSE for "0", "false", "off" and "no"
- Returns NULL otherwise.

```
<?php   $var="yes";

if(filter_var($var, FILTER_VALIDATE_BOOLEAN)) echo " ok";   ?>
```

options :

```php
<?php
// for filters that accept options, use this format
$options = array(
    'options' => array(
   'default' => 3, // value to return if the filter fails
  // other options here
        'min_range' => 0 ,'max_range' => 3
    ),
    'flags' => FILTER_FLAG_ALLOW_OCTAL,
);
$var = filter_var('0755', FILTER_VALIDATE_INT, $options);

// for filter that only accept flags, you can pass them d
irectly
$var = filter_var('oops', FILTER_VALIDATE_BOOLEAN, FILTER
_NULL_ON_FAILURE);

// for filter that only accept flags, you can also pass a
s an array
$var = filter_var('oops', FILTER_VALIDATE_BOOLEAN,
                  array('flags' => FILTER_NULL_ON_FAILURE));
?>
```

Example (**filter_var**):

- Example Passing options to filter_var()

```
<?php
$int_a = '1';  $int_b = '-1';  $int_c = '4';

$options = array( 'options' => array('min_range' => 0,'max_range' => 3,));
if (filter_var($int_a, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "(int_a) integer is considered valid (between 0 and 3).\n";
}
if (filter_var($int_b, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "(int_b) integer is considered valid (between 0 and 3).\n";
}
if (filter_var($int_c, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "(int_c) integer is considered valid (between 0 and 3).\n";
}

$options['options']['default'] = 1;
if(($int_c = filter_var($int_c, FILTER_VALIDATE_INT, $options))
!== FALSE) {
 echo "(int_c) is considered valid (between 0 and 3)and is $int_c.";
}?>
```

- The above example will output:

```
(int_a) integer is considered valid (between 0 and 3).

(int_c) integer is considered valid (between 0 and 3) and is 1
```

Sanitize

- Example: uses the filter_var() function to remove all HTML tags from a string

```php
<?php
$str = "<h1>Hello World!</h1>";
$newstr = filter_var($str, FILTER_SANITIZE_STRING);
echo $newstr;
?>
```

- Example: uses the filter_var() function to remove all illegal characters from the $email variable

```php
<?php  $email = "john.doe@example.com";
// Remove all illegal characters from email
$email = filter_var($email, FILTER_SANITIZE_EMAIL);

// Validate e-mail
if (!filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    echo("$email is a valid email address");
} else {
    echo("$email is not a valid email address");
}
?>
```

Sanitize

EX : Sanitize and Validate a URL

```
 <?php $url = "http://www.w3schools.com";
// Remove all illegal characters from a url
$url = filter_var($url, FILTER_SANITIZE_URL);
// Validate url
if (!filter_var($url, FILTER_VALIDATE_URL) === false) {
    echo("$url is a valid URL");
} else {
    echo("$url is not a valid URL"); } ?>
```

EX : Sanitize and Validate a float

```
 <?php $number="50,000-2f+3.3pp";
echo filter_var($number, FILTER_SANITIZE_NUMBER_FLOAT,
FILTER_FLAG_ALLOW_FRACTION); ?>
```

Sanitize

The FILTER_SANITIZE_STRING filter removes tags and remove or encode special characters from a string. Possible options and flags:

- FILTER_FLAG_NO_ENCODE_QUOTES - Do not encode quotes
- FILTER_FLAG_STRIP_LOW - Remove characters with ASCII value < 32
- FILTER_FLAG_STRIP_HIGH - Remove characters with ASCII value > 127
- FILTER_FLAG_ENCODE_LOW - Encode characters with ASCII value < 32
- FILTER_FLAG_ENCODE_HIGH - Encode characters with ASCII value > 127
- FILTER_FLAG_ENCODE_AMP - Encode the "&" character to &

EX : Sanitize and Validate a string

```
<?php  $str = "<h1>Hello World???!</h1>"; // Variable to check

// Remove HTML tags and all characters with ASCII value > 127

$newstr = filter_var($str, FILTER_SANITIZE_STRING,
FILTER_FLAG_STRIP_HIGH);

echo $newstr; ?>
```

Sanitize

- The FILTER_SANITIZE_ENCODED filter removes or encodes special characters.
- Possible options and flags:
- FILTER_FLAG_STRIP_LOW - Remove characters with ASCII value < 32
- FILTER_FLAG_STRIP_HIGH - Remove characters with ASCII value > 127
- FILTER_FLAG_ENCODE_LOW - Encode characters with ASCII value < 32
- FILTER_FLAG_ENCODE_HIGH - Encode characters with ASCII value > 127

EX : Sanitize and Validate a string using FILTER_SANITIZE_ENCODED

```php
<?php $url="http://www.w3schoolsÅÅ.com"; // Variable to check

// Encode characters
$url = filter_var($url, FILTER_SANITIZE_ENCODED);
echo $url;  ?>
```

//output : http%3A%2F%2Fwww.w3schools%C5%C5.com